

QMST5336: ANALYTICS

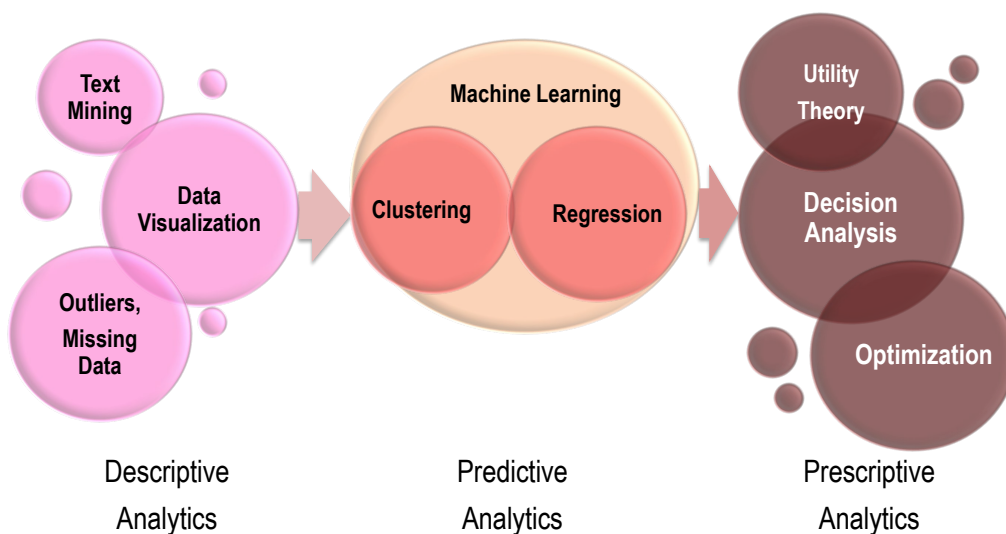
Lecture 4: Text Mining & Vectorization

Emily Zhu, PhD.

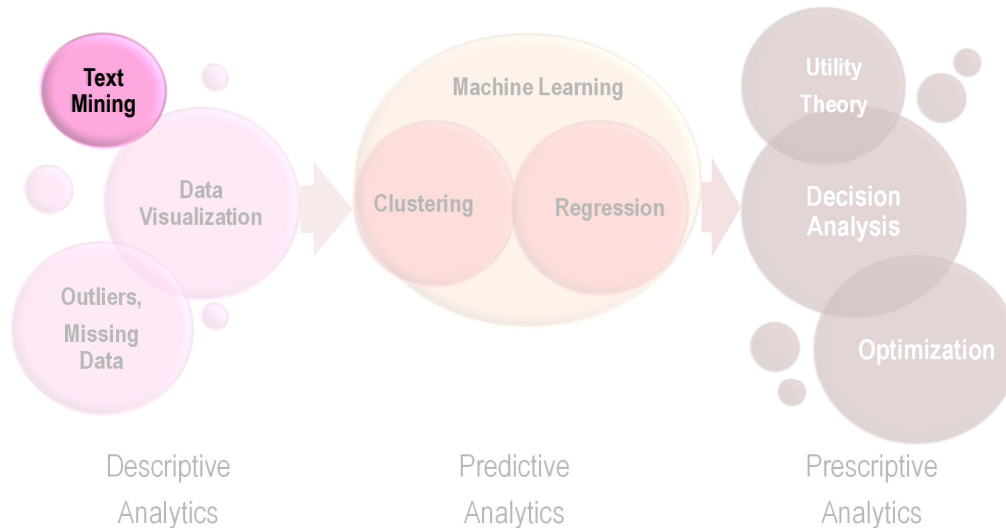
© 2023 Texas State University

Our Problem and Roadmap

An investment property in Austin: **Airbnb** versus **Rental**?



Roadmap: Where We Are



Emily Zhu, PhD.

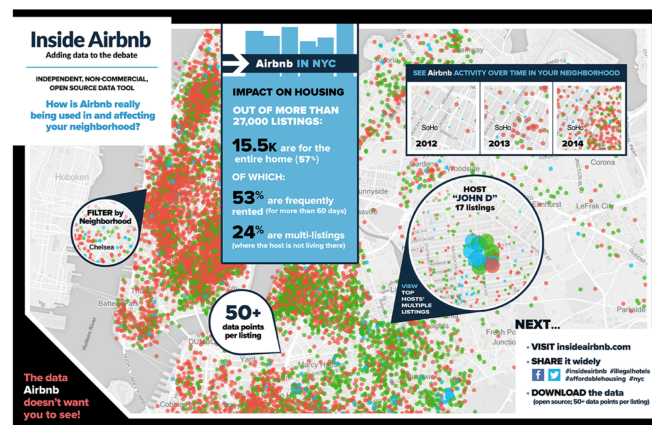
QMST5336: ANALYTICS

© 2023 Texas State University

3 / 49

The Inside AirBnB Information

About Inside Airbnb



11,790 active listings dated in May 2019.

id	neighbourhood	latitude	longitude	property_type	room_type	accommodates	bathrooms	bedrooms	beds	bed_type	amenities	price
1078	University of T	30.30123	-97.73674	Guesthouse	Entire home/apt	2	1	1	1	Real Bed	{TV, Internet, Wifi, "Air conditioning", Kitchen, "	\$85.00
2265	East Downtown	30.2775	-97.71398	House	Entire home/apt	4	2	2	2	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning	\$225.00
5245	East Downtown	30.27577	-97.71379	House	Private room	2	1	1	1	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning	\$100.00
5456	East Downtown	30.26112	-97.73448	Guesthouse	Entire home/apt	3	1	1	2	Real Bed	{TV, Wifi, "Air conditioning", Kitchen, "Pets live or	\$95.00
5769	SW Williamson C	30.45596	-97.7837	House	Private room	2	1	1	1	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning	\$40.00
6413	Travis Heights	30.24829	-97.73726	Guesthouse	Entire home/apt	2	1	1	1	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning	\$99.00
6448	Zilker	30.26027	-97.76503	Guesthouse	Entire home/apt	3	1	1	2	Real Bed	{TV, Internet, Wifi, "Air conditioning", Kitchen, "	\$129.00
8502	East Riverside	30.23616	-97.73682	Guest suite	Entire home/apt	2	1	1	1	Real Bed	{Wifi, "Air conditioning", Kitchen, "Pets live or	\$40.00
14765	Travis Heights	30.25012	-97.74193	House	Entire home/apt	5	2	2	4	Real Bed	{Internet, Wifi, "Air conditioning", "Wheelchair	\$260.00
14913		30.15645	-97.82411	House	Entire home/apt	6	2	3	3	Real Bed	{TV, Internet, Wifi, "Air conditioning", Pool, Kitch	\$300.00
21126	Travis Heights	30.24503	-97.74075	House	Entire home/apt	5	1	2	2	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning	\$550.00
21145	West Campus	30.29422	-97.74618	House	Entire home/apt	4	1	2	3	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning	\$220.00

Process Variables of Listings in Python

id	neighbourhood	latitude	longitude	property_type	room_type	accommodates	bathrooms	bedrooms	beds	bed_type	amenities	price
1078	University of Texas	30.30123	-97.73674	Guesthouse	Entire home/apt	2	1	1	1	Real Bed	{TV, Internet, Wifi, "Air conditioning", Kitchen, "Free parking on premises", Heating, "Smoke detector", Essentials, Shampoo, "24-hour check-in", Hangers, "Hair dryer", Iron, "Laptop friendly workspace", "translation missing: en.hosting_amenity_49", "translation missing: en.hosting_amenity_50", "Self check-in", Keypad}	\$85.00
2265	East Downtown	30.2775	-97.71398	House	Entire home/apt	4	2	2	2	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning", Kitchen, "Free parking on premises", "Paid parking off premises", Breakfast, "Pets live on this property", Dog(s), "Free street parking", Heating, "Family/kid friendly", Washer, Dryer, "Smoke detector", "Carbon monoxide detector", Essentials, Shampoo, Hangers, "Hair dryer", Iron, "Laptop friendly workspace", "Self check-in", Lockbox, "Private entrance", "Hot water", "Bed linens", "Extra pillows and blankets", Microwave, "Coffee maker", Refrigerator, "Dishes and silverware", "Cooking basics", Oven, Stove, "Garden or backyard"}	\$225.00
5245	East Downtown	30.27577	-97.71379	House	Private room	2	1	1	1	Real Bed	{TV, "Cable TV", Internet, Wifi, "Air conditioning", Kitchen, "Free parking on premises", Breakfast, "Pets live on this property", Dog(s), "Free street parking", Heating, "Family/kid friendly", Washer, Dryer, "Smoke detector", "Carbon monoxide detector", Essentials, Shampoo, "Lock on bedroom door", Hangers, "Hair dryer", Iron, "Laptop friendly workspace", "Hot water", "Luggage dropoff allowed", Other}	\$100.00

What should we do with the Variable *amenities*?

Text Mining

Text mining is the process of extracting useful information from text data.

- Text data is often referred to as *unstructured data* because in its raw form, it cannot be stored in a traditional structured database (rows and columns).
 - Audio and video data are also examples of *unstructured data*.
- Text, like numerical data, may contain information that can help solve problems and lead to better decisions.
- Data mining with text data is more challenging than data mining with traditional numerical data, because it requires more preprocessing to convert the text to a format amenable for analysis.

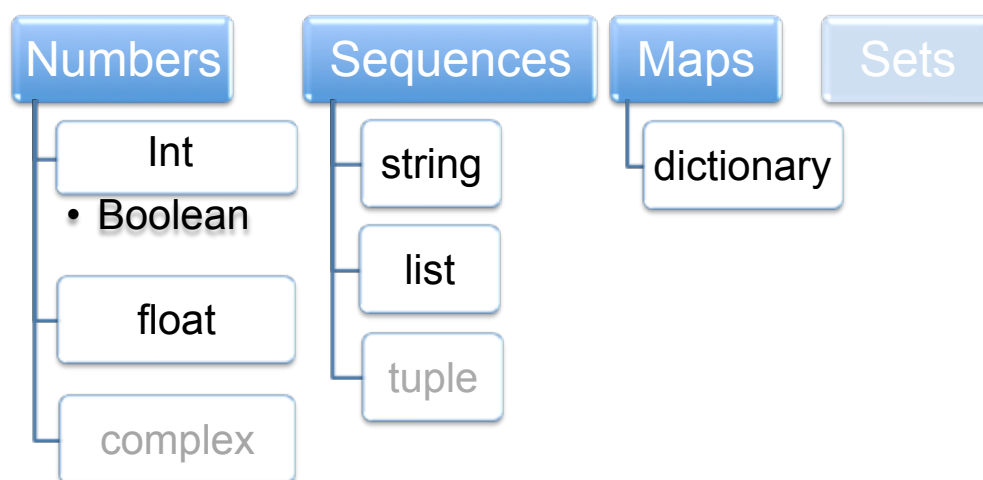
Preprocess Text Data for Analysis

Text-mining process converts unstructured text into numerical data and applies quantitative techniques.

- Frequency can be used to eliminate words from consideration
 - Low-frequency words probably will not be very useful
- Consolidating words that are synonyms
- Different forms of the same word, such as “stacking”, “stacked”, and “stack” probably should not be considered as distinct terms.

Caution: The use of slang, humor, and sarcasm can cause interpretation problems and might require more sophisticated data cleansing and subjective intervention on the part of the analyst to avoid misinterpretation.

Data Types in Python



String

Definition

String is an ordered collection (a sequence) of characters.

You can access the characters one at a time with the bracket operator. The expression in brackets is called an "index". Index has to be an integer, try `fruit[1.2]`

```
1 fruit = "banana"
2 letter = fruit[1]
3 letter
4 # the offset of the first letter is ZERO

'a'
```

Zero Indexing

In Python, the index starts from ZERO by default.

String Slicing

Slice: a segment of a sequence.

- The operator `[n : m]` returns the part of the string from the "n-eth" character to the "m-eth" character, including the first but excluding the last.
- If you omit the first index (before the colon), the slice starts at the beginning of the string.
- If you omit the second index, the slice goes to the end of the string.

```
1 s = "Monty Python"
2 s[:5]
```

'Monty'

```
1 s[5:]
```

' Python'

```
1 s[5:5]
```

' '

String Operators

```
1 new_s = s.upper()
2 new_s
```

'MONTY PYTHON'

```
1 new_s[5:].strip()
```

'PYTHON'

The word `in` is a boolean operator that takes two strings and returns `True` if the first appears as a substring in the second

```
1 "a" in "banana"
```

`True`

```
1 "seed" in "banana"
```

`False`

List

Definition

A List is a sequence of values enclosed in `[]` brackets and separated by commas. List does NOT require the same datatype. The values in a list are called elements or sometimes items.

```
1 L = [2, 3, "fork", -7.8]
2 print(L[0])
```

2

Reference to individual elements of a list: `L[i]` the *i*th element of the list `L`.

This reference method is called indexing "Zero-based" indexing: the first item in a list is numbered 0

```
1 print(L[2])
```

"fork"

```
1 len(L)
```

4

Range

The last element in a list is at position?

```
1 L[len(L) - 1]
```

-7.8

The expression `range(n)` is equivalent to a list containing the numbers 0 through `n-1` in sequence.

```
1 list(range(4))
```

[0, 1, 2, 3]

`range(start, stop, step)` the function creates an object that is equivalent to list obtained by starting to count from `start` by increments of `step`, until it reaches `stop`.

The stop point is always "exclusive", meaning that the first element equal to or beyond the stop point is not part of the resulting sequence. ²

²The `list` function converts the range to an explicitly visible list; without this, we just get (in Python 3.x) a compact range object that behaves in a similar way but uses less memory and does not print very informatively

Range: Forward & Backwards

```
1 list(range(1, 11, 1))
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```
1 range(1, 11, 1)
```

`range(1, 11)`

```
1 #negative steps to count backwards
```

```
2 list(range(10, 0, -1))
```

[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

List are Mutable

```
1 a = list(range(3))
2 b = ['u', 's']
3 a.append(b)
4 a
```

```
[0, 1, 2, ['u', 's']]
```

```
1 a.extend(b)
2 a
```

```
[0, 1, 2, ['u', 's'], 'u', 's']
```

`a.extend(['u', 's'])`

→ `a = [0, 1, 2, 'u', 's']` extended

`a.append(['u', 's'])`

→ `a = [0, 1, 2, ['u', 's']]` nested

Lists versus Strings

```
1 s = 'Game of Thrones'
2 t = list(s)
3 t
```

```
['G', 'a', 'm', 'e', ' ', 'o', 'f', ' ', 'T', 'h', 'r', 'o', 'n', 'e', 's']
```

The string function `string.split()` breaks a string into individual words.

```
1 t = s.split()
2 t
```

```
['Game', 'of', 'Thrones']
```

An optional argument called a delimiter specifies which characters to use as word boundaries. The following example uses a hyphen as a delimiter:

```
1 s = "House-of-cards"
2 delimiter = '-'
3 t = s.split(delimiter)
4 t
```

```
['House', 'of', 'cards']
```

Dictionaries

Definition

A dictionary contains a collection of indices, which are called keys, and a collection of values. Each key is associated with a single value. The association of a key and a value is called a key-value pair or sometimes an item.

A dictionary is like a list, but more general. In a list, the indices have to be integers; in a dictionary they can be (almost) any type.

In mathematical language, a dictionary represents a mapping from keys to values, so you can also say that each key "maps to" a value. As an example, we will build a dictionary that maps from English to numbers, so the keys are all strings and values are numerical.

```
1 eng2n = dict()  
2 eng2n
```

```
1 eng2n['one'] = 1  
2 eng2n
```

```
'one': 1
```

Bonus: get Method for Dictionaries

Dictionaries have a method called `get` that takes a key and a default value. If the key appears in the dictionary, `get` returns the corresponding value; otherwise it returns the default value.

```
1 frq = {'a': 100, 'b': 2, 'D': 4}  
2 f1 = frq['a']  
3 f1
```

```
100
```

```
1 frq.get('a', 0) #0 is the default value
```

```
100
```

```
1 frq.get('c', 0)
```

```
0
```

Case: A Paragraph of GoT Scripts



Daenerys Targaryen: [in Dothraki] Blood of my blood! You . . . [her armies cheer in affirmation. As she watches, Tyrion steps forward to stand beside her. She does not look at him] You freed your brother. You committed treason.

The Iron Throne [8.06] https://gameofthrones.fandom.com/wiki/Season_8

Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

19 / 49

GoT Case: Count Words' Frequency 1

```
1 transcript = "Blood of my blood! You kept all your promises to me.  
    You killed my enemies in their iron suits. You tore down their  
    stone houses. You gave me the Seven Kingdoms! You have walked  
    beside me since the Plaza of Pride. You are the bravest of men,  
    the most loyal of soldiers. I name you commander of all my  
    forces, the Queen's Master of War! Unsullied. All of you were  
    torn from your mothers' arms and made into slaves. Now...you  
    are liberators! You have freed the people of King's Landing  
    from the grip of a tyrant! But the war is not over. We will not  
    lay down our spears until we have liberated all the people of  
    the world! From Winterfell to Dorne, from Lannisport to Qarth,  
    from the Summer Isles to the Jade Sea! Men, women and children  
    have suffered too long beneath the wheel. Will you break the  
    wheel with me?! You freed your brother. You committed treason."  
2  
3 #delimiter = '  
4 l_words = transcript.split()  
5 l_words[:5]
```

```
['Blood', 'of', 'my', 'blood!', 'You']
```

GoT Case: Count Words' Frequency 2

```
1 len(l_words)
```

160

```
1 import string
2 l_goodword = list()
3 for word in l_words:
4     w_1 = word.lower() #make all letters into lower case
5     w_2 = w_1.strip(string.punctuation + string.whitespace)#remove
6     all punctuation and space
7     l_goodword.append(w_2)
8 l_goodword[:5]
```

['blood', 'of', 'my', 'blood', 'you']

```
1 d = dict()
2 for word in l_goodword :
3     d[word] = d.get(word, 0) + 1
4 d
```

GoT Case: Count Words' Frequency 3

```
'blood': 2,
'of': 10,
'my': 3,
'you': 12,
'kept': 1,
'all': 4,
'your': 3,
'promises': 1,
'to': 4,
...
```

```
1 #alternatives
2 d = dict()
3 for word in l_goodword :
4     if word in d :
5         d[word] += 1
6     else :
7         d[word] = 1
8 d
```

GoT Case: Word Cloud



```
1 from wordcloud import WordCloud
2
3 wordcloud = WordCloud(background_color='white',
4                       width=1500, height=1000).generate_from_frequencies(d)
5 plt.figure(figsize=(9,6))
6 plt.imshow(wordcloud)
7 plt.axis('off')
8 plt.show()
```

GoT Case: Visualize Words' Frequency

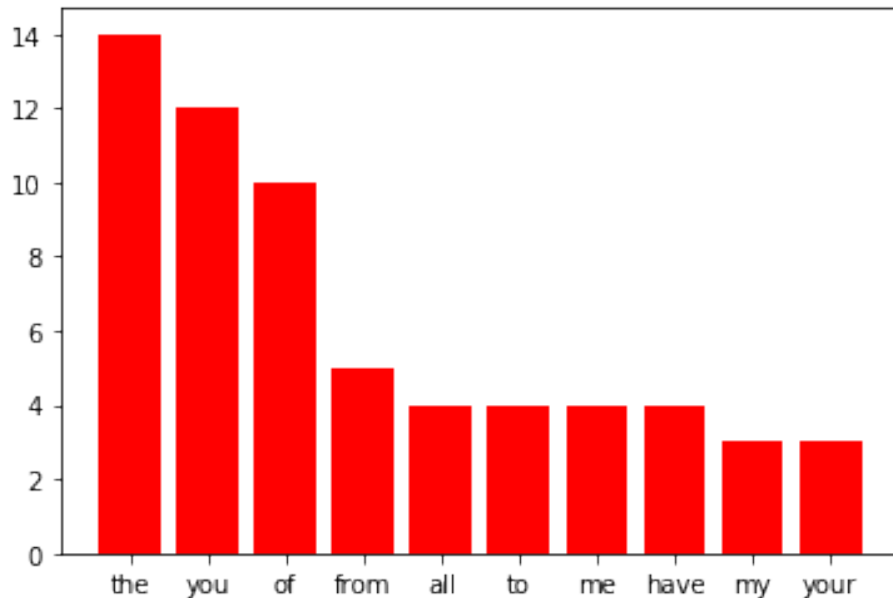
```

1 d_sorted = dict()
2
3 sorted_value = sorted(d.values(), reverse = True)
4 #If True, the sorted list is reversed (or sorted in descending
   order).
5 #Defaults to False if not provided.
6
7 for sortedKey in sorted_value:
8     for key, value in d.items():
9         if value == sortedKey:
10             d_sorted[key] = value
11
12 # Initialize limit
13 K = 10
14
15 # Get first K items in dictionary
16 d_select = dict(list(d_sorted.items())[: K])
17
18 d_select

```

GoT Case: Visualize Words' Frequency C'td

```
1 import matplotlib.pyplot as plt
2
3 plt.bar(list(d_select.keys()), d_select.values(), color='r')
4 plt.show()
```



Count Words' Frequency: Procedure

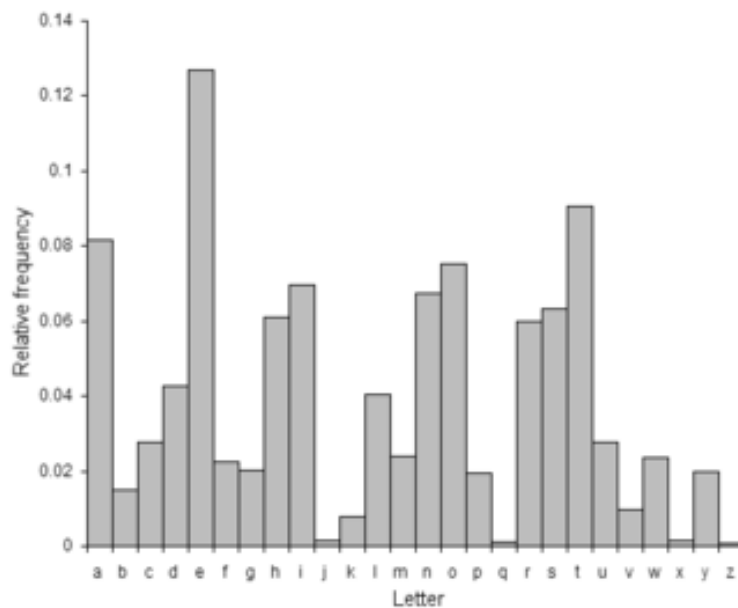
Step 1 Make a list of words by splitting.

Step 2 Covert cases, remove punctuations and spaces.

Step 3 Count frequencies word by word.

Step 4 Visualize words' frequencies.

The Frequency of Alphabet Letters in English



<https://www3.nd.edu/~busiforc/handouts/cryptography/letterfrequencies.html>

Emily Zhu, PhD.

QMST5336: ANALYTICS

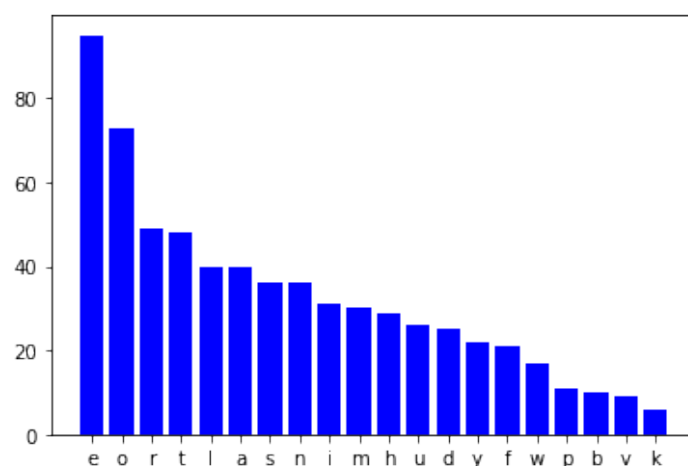
© 2023 Texas State University

27 / 49

GoT Case: Count Letters' Frequency

```
1 #make a list of letters
2 l_letter = []
3 for word in l_words :
4     word = word.lower()
5     word = word.strip(string.punctuation)#remove all punctuations
6     l_letter.extend(list(word))
7 l_letter[:5]
```

['b', 'l', 'o', 'o', 'd']



AirBnb Case: Amenity List

```
1 #import data
2 import os
3 import pandas as pd
4 os.chdir("/Emily/Teaching/Python/Code")
5 df_full = pd.read_csv('df_full_listing.csv', parse_dates = True,
6                       encoding = "utf_8_sig")
7 df_full.columns
8 df_full.shape
```

(11792, 61)

```
1 df = df_full.amenities
2 df.head()
```

```
0 {TV,Internet,Wifi,"Air conditioning",Kitchen,"...
1 {TV,"Cable TV",Internet,Wifi,"Air conditioning...
2 {TV,"Cable TV",Internet,Wifi,"Air conditioning...
3 {TV,Wifi,"Air conditioning",Kitchen,"Pets live...
4 {TV,"Cable TV",Internet,Wifi,"Air conditioning...
Name: amenities, dtype: object
```

AirBnb Case: Amenity List

```
1 df_1 = df.str.split(',')
2 df_1.head()
```

```
0 [{TV, Internet, Wifi, "Air conditioning", Kitc...
1 [{TV, "Cable TV", Internet, Wifi, "Air conditi...
2 [{TV, "Cable TV", Internet, Wifi, "Air conditi...
3 [{TV, Wifi, "Air conditioning", Kitchen, "Pets...
4 [{TV, "Cable TV", Internet, Wifi, "Air conditi...
Name: amenities, dtype: object
```

```
1 l_df = list(df_1)
2 l_word = list()
3 for l_list in l_df:
4     l_word.extend(l_list)
5
6 l_word[:5]
```

```
['{TV', 'Internet', 'Wifi', '"Air conditioning"', 'Kitchen']
```

AirBnb Case: Amenity List C'td

```

1 l_amenity = list() for word in l_word:
2     w_1 = word.strip(string.punctuation + string.whitespace)
3     l_amenity.append(w_1)
4 l_amenity[:5]

['TV', 'Internet', 'Wifi', 'Air conditioning', 'Kitchen']

1 # dictionary
2 d = dict()
3 for word in l_amenity :
4     d[word] = d.get(word, 0) + 1
5 # word cloud
6 wordcloud = WordCloud(background_color='white', width=1500,
7                        height=1000).generate_from_frequencies(d)
8 plt.figure(figsize=(9,6))
9 plt.imshow(wordcloud)
10 plt.axis('off')

```



QMST5336: ANALYTICS

© 2023 Texas State University

31 / 49

AirBnb Case: Neighborhood List

```
1 df = df_full.host_neighbourhood
2 df.head()
```

0 University of Texas

1 East Downtown

2 East Downtown

3 East Downtown

4 SW Williamson Co.

```
Name: host_neighbourhood, dtype: object
```

```
1 df.shape
```

(11792,)

```
1 df_1 = df.dropna()
```

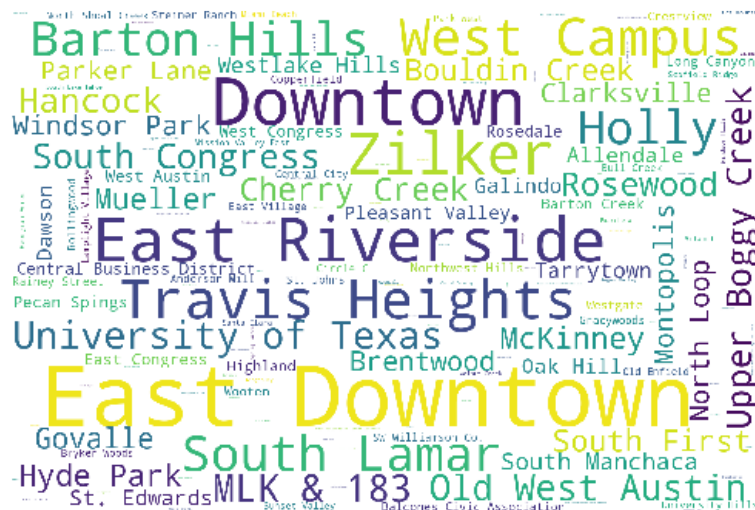
```
2 df.shape
```

(8968,)

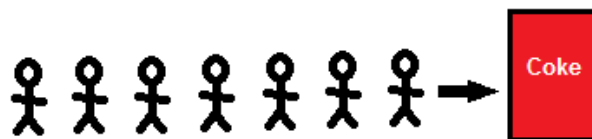
AirBnb Case: Neighborhood List

```
1 l_nbhd = list(df_1)
2 l_nbhd[:5]
```

```
0 ['University of Texas',
'East Downtown',
'East Downtown',
'East Downtown',
'SW Williamson Co.']
```



Vectorization: Computational Efficiency

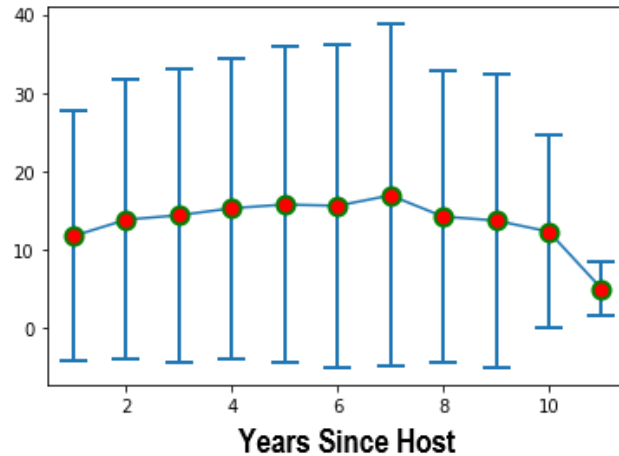


A concurrent solution to an event handling problem: using a queue to resolve conflicts



A parallel solution to a computational problem: avoiding unnecessary conflicts

Case: Numbers of Reviews Across Time



	1	2	3	4	5	6	7	8	9	10	11
1078	11	51	46	47	50	0	13	0	0	0	0
2265	2	0	1	2	3	1	1	6	3	3	2
5245	1	1	1	0	0	1	2	0	2	1	0
5456	36	58	47	37	46	48	54	51	43	51	10
5769	0	21	17	30	53	33	30	14	18	23	7

The Data: from Two Spreadsheets

Start time per listing

`df_full_listings.csv`

`df_listing.head()`

	id	host_since
0	1078	2013-01-08
1	2265	2008-08-23
2	5245	2008-08-23
3	5456	2009-02-16
4	5769	2009-02-19

Times of Reviews

`reviews.csv`

`df_full.head()`

	listing_id	date
0	1078	2008-09-13
1	1078	2011-03-22
2	1078	2011-04-04
3	1078	2013-08-26
4	1078	2013-09-05

Pandas Operations on datetime



Change to datetime Type

```
1 df_full.date = pd.to_datetime(df_full.date)
2 df_full.date.head()
```

0 2008-09-13

1 2011-03-22

2 2011-04-04

3 2013-08-26

4 2013-09-05

Name: date, dtype: datetime64[ns]

```
1 #Set listing ID as index
2 df_listing.set_index('id', inplace=True, drop=True)
3 df_listing.head()
```

	host_since
id	
1078	2013-01-08
2265	2008-08-23
5245	2008-08-23
5456	2009-02-16
5769	2009-02-19

A Few Setups

```
1 #Years as column names
2 Yr = np.arange(1, 12, 1)
3 print(Yr)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11]
```

```
1 #Record listing ID to be used as index
2 narray_id = df_full.listing_id.unique()
3 narray_id[-5:]
```

```
array([34538859, 34567478, 34569032, 34574163, 34600376])
```

```
1 #the table to be filled
2 df_nrv1 = pd.DataFrame(0, columns = Yr, index = narray_id)
3 df_nrv1.head()
```

	1	2	3	4	5	6	7	8	9	10	11
1078	0	0	0	0	0	0	0	0	0	0	0
2265	0	0	0	0	0	0	0	0	0	0	0
5245	0	0	0	0	0	0	0	0	0	0	0
5456	0	0	0	0	0	0	0	0	0	0	0
5769	0	0	0	0	0	0	0	0	0	0	0

Method 1: Please Do Avoid

```
1 #Method 1: 2 Loops
2 #PLEASE TRY TO AVOID
3 df_nrv1 = pd.DataFrame(0, columns = Yr, index = narray_id)
4 for ii in df_nrv1.index:
5     if not pd.isnull(df_listing.host_since[ii]):
6         df_rr = df_full[df_full.listing_id == ii]
7         for jj in range(len(df_rr)):
8             # type Timedelta
9             l_interval = df_rr.iloc[jj]['date'] - df_listing.
host_since[ii]
10             # convert Timedelta to numbers
11             n_interval = pd.to_numeric(l_interval.days, errors = '
coerce')
12             n_yr = math.ceil(n_interval/365)
13             if n_yr > 0 and n_yr < 12:
14                 df_nrv1.loc[ii, n_yr] += 1
15 df_nrv1.head()
```

	1	2	3	4	5	6	7	8	9	10	11
1078	11	51	46	47	50	0	13	0	0	0	0
2265	2	0	1	2	3	1	1	6	3	3	2
5245	1	1	1	0	0	1	2	0	2	1	0
5456	36	58	47	37	46	48	54	51	43	51	10
5769	0	21	17	30	53	33	30	14	18	23	7

Method 2: Remove Inner Loop

```
1 #Method 2: Remove Inner Loop
2 df_nrv2 = pd.DataFrame(0, columns = Yr, index = narray_id)
3 bin_range = np.arange(12)
4 for ii in df_nrv2.index:
5     if not pd.isnull(df_listing.host_since[ii]):
6         df_rr = df_full[df_full.listing_id == ii]
7         df_interval = df_rr.date - df_listing.host_since[ii]
8         df_n_days = pd.to_numeric(df_interval.dt.days)
9         df_n_years = df_n_days/365
10        out = pd.cut(df_n_years, bins = bin_range, include_lowest =
11        True, right = False)
12        df_nrv2.loc[ii] = out.value_counts(sort = False).values
df_nrv2.head()
```

	1	2	3	4	5	6	7	8	9	10	11
1078	11	51	46	47	50	0	13	0	0	0	0
2265	2	0	1	2	3	1	1	6	3	3	2
5245	1	1	1	0	0	1	2	0	2	1	0
5456	36	58	47	37	46	48	54	51	43	51	10
5769	0	21	17	30	53	33	30	14	18	23	7

CPU times: user 1min 13s, sys: 313 ms, total: 1min 14s

Wall time: 1min 14s

Method 3: Use apply()

```
1 #Method 3: Use Apply()
2 def fun_count_yearly(x):
3     df_rr = df_full[df_full.listing_id == x]
4     if not pd.isnull(df_listing.host_since[x]):
5         df_interval = df_rr.date - df_listing.host_since[x]
6         df_n_days = pd.to_numeric(df_interval.dt.days)
7         df_n_years = df_n_days/365
8         out = pd.cut(df_n_years, bins = bin_range, include_lowest =
9         True, right = False)
10        return out.value_counts(sort = False).values
11
12 l_nrv3 = pd.Series(narray_id).apply(fun_count_yearly)
13 df_nrv3 = l_nrv3.apply(pd.Series)
14 df_nrv3.index = narray_id
15 df_nrv3.columns = Yr
df_nrv3.head()
```

	1	2	3	4	5	6	7	8	9	10	11
1078	11.0	51.0	46.0	47.0	50.0	0.0	13.0	0.0	0.0	0.0	0.0
2265	2.0	0.0	1.0	2.0	3.0	1.0	1.0	6.0	3.0	3.0	2.0
5245	1.0	1.0	1.0	0.0	0.0	1.0	2.0	0.0	2.0	1.0	0.0
5456	36.0	58.0	47.0	37.0	46.0	48.0	54.0	51.0	43.0	51.0	10.0
5769	0.0	21.0	17.0	30.0	53.0	33.0	30.0	14.0	18.0	23.0	7.0

CPU times: user 1min 9s, sys: 229 ms, total: 1min 9s

Wall time: 1min 9s

Method 4: Don't Forget NumPy

```
1 #Method 4: Don't Forget NumPy
2 def fun_count_yearly(x):
3     df_rr = df_full[df_full.listing_id == x]
4     if not pd.isnull(df_listing.host_since[x]):
5         df_interval = df_rr.date - df_listing.host_since[x]
6         df_n_days = pd.to_numeric(df_interval.dt.days)
7         df_n_years = df_n_days/365
8         out = np.histogram(df_n_years, bins = bin_range)
9         return out[0]
10
11 l_nrv4 = pd.Series(narray_id).apply(fun_count_yearly)
12 df_nrv4 = l_nrv3.apply(pd.Series)
13 df_nrv4.index = narray_id
14 df_nrv4.columns = Yr
15 df_nrv4.head()
```

	1	2	3	4	5	6	7	8	9	10	11
1078	11.0	51.0	46.0	47.0	50.0	0.0	13.0	0.0	0.0	0.0	0.0
2265	2.0	0.0	1.0	2.0	3.0	1.0	1.0	6.0	3.0	3.0	2.0
5245	1.0	1.0	1.0	0.0	0.0	1.0	2.0	0.0	2.0	1.0	0.0
5456	36.0	58.0	47.0	37.0	46.0	48.0	54.0	51.0	43.0	51.0	10.0
5769	0.0	21.0	17.0	30.0	53.0	33.0	30.0	14.0	18.0	23.0	7.0

CPU times: user 24.4 s, sys: 82.9 ms, total: 24.5 s

Wall time: 24.4 s

Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

43 / 49

Method 5: Vectorization in 2-Ds

```
1 #Method 5: Vectorization
2 df_inner = pd.merge(df_full, df_listing, left_on = 'listing_id',
3                     right_on = 'id', how = 'inner')
4 df_interval = df_inner.date - df_inner.host_since
5 df_days = pd.to_numeric(df_interval.dt.days)
6 df_inner['#Yr'] = df_days/365
7
8 bin_0 = df_inner.listing_id.unique()
9 bin_1 = np.append(bin_0, np.max(bin_0) + 1)
10 bin_range = np.arange(12)
11 out = np.histogram2d(df_inner.listing_id, df_inner['#Yr'], bins = [
12     bin_1, bin_range])
13
14 df_nrv5 = pd.DataFrame(out[0], columns = Yr, index = bin_0)
15 df_nrv5.head()
```

	1	2	3	4	5	6	7	8	9	10	11
1078	11.0	51.0	46.0	47.0	50.0	0.0	13.0	0.0	0.0	0.0	0.0
2265	2.0	0.0	1.0	2.0	3.0	1.0	1.0	6.0	3.0	3.0	2.0
5245	1.0	1.0	1.0	0.0	0.0	1.0	2.0	0.0	2.0	1.0	0.0
5456	36.0	58.0	47.0	37.0	46.0	48.0	54.0	51.0	43.0	51.0	10.0
5769	0.0	21.0	17.0	30.0	53.0	33.0	30.0	14.0	18.0	23.0	7.0

CPU times: user 263 ms, sys: 42.2 ms, total: 306 ms

Wall time: 301 ms

Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

44 / 49

Coding Breakdown

```
1 df_inner = pd.merge(df_full, df_listing, left_on = 'listing_id',
    right_on = 'id', how = 'inner')
2 df_inner.head()
```

	listing_id	date	host_since
0	1078	2008-09-13	2013-01-08
1	1078	2011-03-22	2013-01-08
2	1078	2011-04-04	2013-01-08
3	1078	2013-08-26	2013-01-08
4	1078	2013-09-05	2013-01-08

```
1 df_inner.listing_id.nunique()
```

9273

```
1 bin_0 = df_inner.listing_id.unique()
2 bin_1 = np.append(bin_0, np.max(bin_0) + 1)
3 bin_1
```

array([1078, 2265, 5245, ..., 34574163, 34600376, 34600377])

The Output As An Array

```
1 bin_range = np.arange(12)
2 out = np.histogram2d(df_inner.listing_id, df_inner['#Yr'], bins = [
    bin_1, bin_range])
3 out[1] #bins of listing IDs, i.e. bin_1
```

array([1078, 2265, 5245, ..., 34574163, 34600376, 34600377])

```
1 out[2] #bins years, i.e. bin_range
```

array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])

```
1 out[0]
```

array([[11., 51., 46., ..., 0., 0., 0.],
 [2., 0., 1., ..., 3., 3., 2.],
 [1., 1., 1., ..., 2., 1., 0.],
 ...,
 [0., 0., 1., ..., 0., 0., 0.],
 [0., 0., 1., ..., 0., 0., 0.],
 [0., 0., 0., ..., 1., 0., 0.]])

```
1 out[0].shape #out[0] is a numpy ndarray
```

(9273, 11)

Why Vectorization?

Method	Time
2 loops	> 3.5 mins
1 loop + pandas	1.25 mins
apply + pandas	1 min 9s
apply + NumPy	24.4s
2-D vectorization	263 ms

The Preferences:

- 1 NumPy
- 2 pandas for functions of the same function
- 3 apply
- 4 should always avoid loop(s)
- 5 not progress forward



Takeaway: Vectorization

Work Smarter NOT Harder



Stand On a Giant's Shoulder



Next Lecture

- **Assignment 1** due date 11:59PM Sep. 20
- **Group Meeting** requires attendance
 - Sep. 21

